

TEMA 8-E: APRENDIZAJE NO SUPERVISADO

Los métodos de aprendizaje no supervisados tratan problemas en los que los datos etiquetados no están disponibles. No se puede obtener ninguna forma de retroalimentación supervisada sobre el procesamiento que ocurre. La ausencia de muestras etiquetadas marca un cambio fundamental de pensamiento de los otros métodos supervisados discutidos hasta ahora. También podría parecer que tal procesamiento podría no producir ninguna información útil, pero eso no es así.

Hay muchas situaciones en las que los métodos no supervisados son extremadamente valiosos. El primero y más importante es el costo del etiquetado. En muchas situaciones, tener todos los datos de entrenamiento completamente etiquetados puede ser bastante costoso y prácticamente imposible y es necesario trabajar con un pequeño conjunto de datos etiquetados. En tales casos, se puede comenzar con métodos supervisados utilizando un pequeño conjunto de datos etiquetados y luego hacer crecer el modelo sin supervisión en un conjunto más grande de datos. En otros casos, es posible que las etiquetas no estén disponibles en absoluto, y es necesario comprender la estructura y composición de los datos realizando un análisis exploratorio sin supervisión. Comprender la estructura de los datos también puede ayudar a elegir el algoritmo correcto o un mejor conjunto de parámetros inicializados para el algoritmo en los casos en que se utilizarán en última instancia métodos supervisados.

K-MEANS

El agrupamiento k-means es uno de los algoritmos de agrupamiento más simples y más utilizados. Intenta encontrar centros de conglomerados que sean representativos de ciertas regiones de los datos. El algoritmo alterna entre dos pasos: asignar cada punto de datos al centro de conglomerado más cercano y luego establecer cada centro de conglomerado como la media de los puntos de datos que se le asignan. El algoritmo finaliza cuando la asignación de instancias a los clústeres ya no cambia.

En la figura de la transparencia se muestra la evolución del algoritmo en un conjunto de datos sintético. Los centros de conglomerados se muestran como triángulos, mientras que los puntos de datos se muestran como círculos. Los colores indican la pertenencia al clúster. Especificamos que buscamos tres conglomerados (ese sería el valor de K), por lo que el algoritmo se inicializó declarando tres puntos de datos al azar como centros de conglomerados. Entonces comienza el algoritmo iterativo. Primero, cada punto de datos se asigna al centro del conglomerado al que está más cercano. A continuación, los centros de conglomerados se actualizan para que sean la media de los puntos asignados. Luego se repite el proceso dos veces más. Después de la tercera iteración, la asignación de puntos a los centros de los conglomerados permaneció sin cambios, por lo que el algoritmo se detiene. Dados los nuevos puntos de datos, k-means asignará cada uno al centro del grupo más cercano.

La figura de la derecha muestra los límites de los centros de conglomerados que se aprendieron. Pueden ver que el agrupamiento es algo similar a la clasificación, en el sentido de que cada elemento recibe una etiqueta. Sin embargo, no existe una verdad básica previa y, en consecuencia, las etiquetas en sí mismas no tienen un significado a priori. Para el agrupamiento que acabamos de calcular en el conjunto de datos de juguete, eso significa que no debemos asignar ningún significado al hecho de que un grupo se haya etiquetado como 0 y otro como 1.

Valor de k

La dificultad que tiene asociada este algoritmo es la determinación del valor de k , es decir, de antemano debemos decirle al algoritmo la cantidad de clusters que debe encontrar. Para realizar esto de una forma más sistemática tenemos el método del codo.

El método del codo, también conocido como "método de la suma de cuadrados intra-cluster" (WCSS, por sus siglas en inglés), es una técnica utilizada en análisis de clúster para determinar el número óptimo de clústeres en un conjunto de datos. El objetivo es encontrar un equilibrio entre la calidad de la agrupación (clústeres compactos) y el número de clústeres, evitando un número excesivo o insuficiente de grupos.

Se comienza ejecutando el algoritmo K-Means con diferentes valores de " k ". Podemos variar " k " desde 1 hasta un número razonable, dependiendo de nuestro problema y la cantidad de datos. Para cada valor de " k ", K-Means agrupa los datos en " k " clústeres y calcula la suma de cuadrados intra-cluster, que representa la suma de los cuadrados de las distancias euclidianas entre cada punto de datos y el centro del clúster al que pertenece. En otras palabras, mide la dispersión de los datos dentro de los clústeres. Cuanto menor sea este valor, más compactos y cohesivos serán los clústeres.

Luego, se crea un gráfico que muestra los valores de " k " en el eje horizontal y los valores correspondientes a la suma de cuadrados intra-cluster en el eje vertical. El "codo" en el gráfico es el punto en el que la disminución de la suma de cuadrados intra-cluster comienza a aplanarse, formando una curva que se asemeja a la forma de un codo. El punto del codo sugiere un equilibrio entre la calidad de la agrupación y el número de clústeres. En general, se busca el valor de " k " en el punto del codo.

Tengan en cuenta que el método del codo es una técnica heurística y a veces puede no ser claro o tener un codo muy pronunciado. En tales casos, es posible que tengan que utilizar otras métricas y enfoques para validar la elección.

Dificultades

Incluso si conocemos la cantidad "correcta" de grupos para un conjunto de datos determinado, es posible que k-means no siempre pueda recuperarlos. Cada grupo se define únicamente por su centro, lo que significa que cada grupo tiene una forma convexa. Como resultado de esto, k-means solo puede capturar formas relativamente simples. K-means también supone que todos los grupos tienen el mismo diámetro en algún sentido; siempre dibuja el límite entre los grupos para que esté exactamente en el medio entre los centros de los grupos. Eso a veces puede conducir a resultados sorprendentes.

Por ejemplo, en el primer caso de la transparencia, uno podría haber esperado que la región densa en la parte inferior izquierda fuera el primer grupo, la región densa en la parte superior derecha fuera el segundo y la región menos densa en el centro fuera el tercero. En cambio, tanto el cluster 0 como el cluster 1 tienen algunos puntos que están lejos de todos los demás puntos de estos conglomerados que "llegan" hacia el centro.

K-means también asume que todas las direcciones son igualmente importantes para cada grupo. La siguiente gráfica muestra un conjunto de datos bidimensional donde hay tres partes claramente separadas en los datos. Sin embargo, estos grupos se estiran hacia la diagonal. Como k-means solo considera la distancia al centro del clúster más cercano, no puede manejar este tipo de datos.

K-means también funciona mal si los grupos tienen formas más complejas, como los datos en forma de lunas. Aquí, esperamos que el algoritmo de agrupamiento pueda descubrir las dos formas de media luna. Sin embargo, esto no es posible usando el algoritmo k-means.

En la parte inferior de la transparencia, en el lado izquierdo, la agrupación intuitiva de los datos es en forma de un pequeño anillo rodeado por uno más grande. Al lado, vemos los mismos puntos de datos agrupados por el algoritmo K Means, con un valor $K=2$, donde cada centroide se representa con una forma de diamante. Como pueden ver, el algoritmo no identifica la agrupación intuitiva.

En la última imagen vemos la agrupación de dos grupos de datos reconocibles. Uno pequeño pintado de celeste y otro mayor de rojo. Al lado vemos el resultado de la agrupación de K-means sobre los mismos puntos de datos que no encaja con la agrupación intuitiva. Como en el caso del ejemplo anterior, K Means creó particiones que no reflejan lo que visualmente identificamos debido a la limitación esférica del algoritmo. Intenta encontrar centroides con esferas de datos ordenados a su alrededor, y funciona mal ya que la forma geométrica del clúster se desvía de una esfera.

Veamos ahora cómo implementar este algoritmo utilizando la librería Scikit Learn. El primer paso que tenemos que hacer es importar el modulo Kmeans desde `sklearn.cluster`. Recuerden que este es un algoritmo de agrupamiento por lo que es normal que se encuentre dentro del módulo “cluster”.

Realizado esto analizamos los parámetros que podemos implementar junto con el algoritmo Kmeans, estos no son muy distintos a los que se aplican dentro de los algoritmos de Aprendizaje Supervisado, pero la forma de implementarlos es un poco distinta.

El primero será `fit()`, con el cual podemos entrenar el modelo. Acá solamente debemos utilizar los datos correspondientes a X, que vendrían a ser los datos de entrenamiento. Para los algoritmos de Aprendizaje no Supervisado no se utiliza valores de “y” o lo que serían los datos dependientes o etiquetas ya que acá nos enfocamos básicamente en entender los datos más que predecir un valor específico como se hace en el Aprendizaje Supervisado.

El siguiente método será `predict()`, con el cual podemos realizar una predicción, determinando los clústeres más cercanos al que pertenece cada muestra en X. En este caso podemos seleccionar unas muestras o en todo caso nueva data, para determinar a que clúster pertenece de acuerdo al modelo que se ha entrenado previamente.

El resultado que tenemos al implementar este algoritmo será el número de clúster asociado a cada dato del conjunto de datos.

El primer parámetro a definir por supuesto será el número de clústeres. El siguiente parámetro a definir será ***init*** y se refiere al método de inicialización. Este parámetro puede tomar uno de estos 3 métodos, `k-means++` (que selecciona los centroides iniciales de los clústeres de forma inteligente para acelerar la convergencia), `random` o `ndarray` (donde se dan específicamente los centroides), por defecto el que se encuentra configurado es `k-means++`.

El siguiente parámetro a configurar para el algoritmo K Means será ***n_init***, este se refiere al número de veces que el algoritmo se ejecutará con diferentes centroides. Este parámetro tiene como valor por defecto de 10 pero podemos configurarlo para verificar como mejora o empeora el modelo.

El último parámetro que es importante configurar será ***max_iter***, este es el número máximo de iteraciones del algoritmo K Means para una sola ejecución. Este sería el criterio de parada. Por defecto el valor de este parámetro es de 300 pero lo podemos disminuir e inclusive aumentar si es necesario.

AGRUPACIÓN AGLOMERATIVA

El agrupamiento aglomerativo se refiere a una colección de algoritmos de agrupamiento que se basan en los mismos principios: el algoritmo comienza declarando cada punto como su propio grupo y luego fusiona los dos grupos más similares hasta que se satisface algún criterio de parada. El criterio de detención implementado en `scikit-learn` es la cantidad de clústeres, por lo que los clústeres similares se fusionan hasta que solo queda la cantidad especificada de clústeres.

Hay varios criterios de vinculación que especifican cómo se mide exactamente el "grupo más similar". Las tres opciones que se implementan en scikit-learn son:

- a) La opción predeterminada, **Ward**, elige los dos grupos para fusionarlos de modo que la varianza dentro de todos los grupos aumente lo mínimo. Esto a menudo conduce a clústeres que tienen un tamaño relativamente similar.
- b) El **enlace promedio** fusiona los dos grupos que tienen la distancia promedio más pequeña entre todos sus puntos.
- c) Y el **enlace completo** (también conocido como enlace máximo) fusiona los dos grupos que tienen la distancia máxima más pequeña entre sus puntos.

Ward funciona bien en la mayoría de los conjuntos de datos. Si los clústeres tienen un número muy diferente de miembros (si uno es mucho más grande que todos los demás, por ejemplo), el promedio o el completo podrían funcionar mejor.

DENDROGRAMAS

A veces es útil examinar todas las agrupaciones posibles de forma conjunta. En la Figura de la izquierda de la transparencia vemos una superposición de todos los agrupamientos posibles, lo que proporciona una idea de cómo cada grupo se divide en grupos más pequeños.

Si bien esta visualización proporciona una vista muy detallada del agrupamiento jerárquico, se basa en la naturaleza bidimensional de los datos y, por lo tanto, no se puede usar en conjuntos de datos que tienen más de dos características. Sin embargo, existe otra herramienta para visualizar la agrupación jerárquica, llamada dendrograma, que puede manejar conjuntos de datos multidimensionales.

El dendrograma muestra puntos de datos como puntos en la parte inferior (numerados del 0 al 11). Luego, se traza un árbol con estos puntos (que representan grupos de un solo punto) como las hojas, y se agrega un nuevo padre de nodo para cada dos grupos que se unen.

Leyendo de abajo hacia arriba, los puntos de datos 1 y 4 se unen primero (como se puede ver en la Figura). A continuación, los puntos 6 y 9 se unen en un grupo, y así sucesivamente. En el nivel superior hay dos ramales, uno formado por los puntos 11, 0, 5, 10, 7, 6 y 9, y otro formado por los puntos 1, 4, 3, 2 y 8. Estos corresponden a los dos grupos más grandes en la figura del lado izquierdo.

La longitud de cada rama en el dendrograma también muestra qué tan separados están los grupos combinados. Las ramas más largas en este dendrograma son las tres líneas que están marcadas por la línea discontinua etiquetada como "tres grupos". Que estas sean las ramas más largas indica que pasar de tres a dos grupos significó fusionar algunos puntos muy alejados. Vemos esto nuevamente en la parte superior del gráfico, donde la fusión de los dos grupos restantes en un solo grupo nuevamente une una distancia relativamente grande.

Desafortunadamente, el agrupamiento aglomerativo aún falla al separar formas complejas como el conjunto de datos en medialuna que vimos en el caso de KMeans. Sin embargo, no ocurre lo mismo con el siguiente algoritmo que veremos, DBSCAN.

Con respecto a los parámetros en la implementación de scikit learn el primero a definir, por supuesto, será el número de clústeres. Por defecto este valor es de 2 en caso de que no sea definido. El siguiente parámetro es "**affinity**" que se refiere a las medidas de distancia que se utilizarán para calcular la vinculación. Esta métrica puede ser "euclidean", referente a la distancia euclidiana, "manhattan" referente a la distancia manhattan, "cosine" referente a la distancia del coseno o "precomputed".

Finalmente, el siguiente parámetro a definir es "**linkage**" o las medidas de vinculación. El criterio de vinculación determina qué distancia usar entre los conjuntos de observación. El algoritmo fusionará

los pares de cúmulos que minimizan este criterio. Las medidas de vinculación que se pueden seleccionar son: ward, la cual se encuentra configurada por defecto, "complete" haciendo referencia al enlace completo, "average" refiriéndose al enlace promedio.

DBSCAN

Otro algoritmo de agrupamiento muy útil es DBSCAN (que significa "agrupamiento espacial de aplicaciones con ruido basado en la densidad"). Los principales beneficios de DBSCAN son que no requiere que el usuario establezca el número de conglomerados a priori, puede capturar conglomerados de formas complejas y puede identificar puntos que no forman parte de ningún conglomerado. DBSCAN es un poco más lento que el agrupamiento aglomerativo y k-means, pero se adapta a conjuntos de datos relativamente grandes.

DBSCAN funciona mediante la identificación de puntos que se encuentran en regiones "pobladas" del espacio de características, donde muchos puntos de datos están muy juntos. Estas regiones se denominan regiones densas en el espacio de características. La idea detrás de DBSCAN es que los grupos forman regiones densas de datos, separados por regiones que están relativamente vacías.

Los puntos que se encuentran dentro de una región densa se denominan muestras centrales (o puntos centrales) y se definen de la siguiente manera. Hay dos parámetros en DBSCAN: *min_samples* y *eps*. Si hay al menos *min_samples* dentro de una distancia de *eps* a un punto de datos dado, ese punto de datos se clasifica como una muestra central.

El algoritmo funciona eligiendo un punto arbitrario para comenzar. Luego encuentra todos los puntos con distancia *eps* o menos desde ese punto. Si hay menos de *min_samples* puntos a una distancia *eps* del punto de inicio, este punto se etiqueta como ruido, lo que significa que no pertenece a ningún grupo. Si hay más de *min_samples* puntos dentro de una distancia de *eps*, el punto se etiqueta como muestra central y se le asigna una nueva etiqueta de grupo. Luego, se visitan todos los vecinos (dentro de *eps*) del punto. Si aún no se les ha asignado un clúster, se les asigna la nueva etiqueta de clúster que se acaba de crear. Si son muestras de núcleo, sus vecinos son visitados a su vez, y así sucesivamente. El grupo crece hasta que no hay más muestras de núcleo dentro de una distancia *eps* del grupo. Luego se selecciona otro punto que aún no ha sido visitado y se repite el mismo procedimiento.

Al final, hay tres tipos de puntos: puntos centrales, puntos que están a una distancia *eps* de los puntos centrales (llamados puntos límite) y ruido. Cuando el algoritmo DBSCAN se ejecuta en un conjunto de datos en particular varias veces, la agrupación de los puntos centrales siempre es la misma y los mismos puntos siempre se etiquetarán como ruido. Sin embargo, un punto límite puede ser vecino de muestras centrales de más de un conglomerado. Por lo tanto, la pertenencia a un grupo de puntos límite depende del orden en que se visitan los puntos. Por lo general, solo hay unos pocos puntos límite, y esta ligera dependencia del orden de los puntos no es importante.

El parámetro *eps* es importante, ya que determina lo que significa que los puntos estén "cerca". Establecer *eps* para que sea muy pequeño significará que ningún punto es una muestra central y puede llevar a que todos los puntos se etiqueten como ruido. Establecer *eps* para que sea muy grande dará como resultado que todos los puntos formen un solo grupo.

La configuración de *min_samples* determina principalmente si los puntos en regiones menos densas se etiquetarán como valores atípicos o como sus propios grupos. Si disminuye *min_samples*, cualquier cosa que hubiera sido un grupo con menos de *min_samples* ahora se etiquetará como ruido. *min_samples*, por lo tanto, determina el tamaño mínimo del clúster.

Si bien DBSCAN no requiere establecer la cantidad de clústeres explícitamente, configurar *eps* controla implícitamente cuántos clústeres se encontrarán. Encontrar una buena configuración para *eps* a veces es más fácil después de escalar los datos con *StandardScaler* o *MinMaxScaler*, ya que el uso de estas técnicas de escalado garantizará que todas las funciones tengan rangos similares.

Con respecto a los parámetros en *scikit learn*, el primero a definir será ***eps*** o *épsilon*, que es la distancia máxima entre dos muestras para que una sea considerada como el clúster de la otra. Este es el parámetro más importante de DBSCAN por lo que se debe elegir apropiadamente para el conjunto de datos y la función de distancia que se está trabajando. Por defecto, este valor es de 0,5 pero se puede modificar por cualquier otro número flotante.

El siguiente parámetro es ***mínimo de muestras*** o puntos, este será el número mínimo de puntos para formar un clúster. Por defecto este valor es de 5 pero obviamente podemos cambiarlo por cualquier otro valor entero.

El último parámetro que normalmente se configura para este algoritmo es el de ***métrica***, acá se define el tipo de métrica a utilizar cuando se calcula la distancia entre los puntos de un conjunto de características. Por defecto, este parámetro es euclidiano, pero podemos configurarlo con alguna métrica “precalculada”.

EVALUACIÓN

Uno de los desafíos en la aplicación de algoritmos de agrupamiento es que es muy difícil evaluar qué tan bien funcionó un algoritmo y comparar los resultados entre diferentes algoritmos. Hay métricas que se pueden usar para evaluar el resultado de un algoritmo de agrupamiento en relación con un agrupamiento de campo, siendo los más importantes el índice de rand ajustado (ARI) y contorno de silueta.

El Índice de Rand Ajustado (ARI) evalúa la similitud entre dos agrupaciones (por ejemplo, clústeres predichos vs. etiquetas reales o entre dos soluciones de agrupamiento). Compara qué tan bien coinciden las agrupaciones con lo que se esperaría por azar. Rango de valores que devuelve:

1: Coincidencia perfecta.

0: Coincidencia aleatoria.

Valores negativos: Peor que el azar.

El Índice de Silueta evalúa qué tan bien están separados los clústeres y qué tan compactos son los puntos dentro de cada clúster. Compara la distancia promedio de un punto a otros puntos en su propio clúster con la distancia promedio a puntos en otros clústeres. Rango de valores que devuelve:

1: Los clústeres están bien separados y los puntos están bien agrupados.

0: Los puntos están en el límite entre dos clústeres.

Negativo: Los puntos podrían estar en el clúster equivocado.

En la figura vemos una comparación de los algoritmos k-means, agrupamiento aglomerativo y DBSCAN usando ARI. También incluimos cómo se ve cuando asignamos puntos aleatoriamente a dos grupos para comparar.

El índice de rand ajustado proporciona resultados intuitivos, con una asignación de clúster aleatoria con una puntuación de 0 y DBSCAN (que recupera perfectamente la agrupación deseada) con una puntuación de 1.

Con respecto a la puntuación de silueta, como podemos ver, k-means obtiene la puntuación de silueta más alta, aunque preferimos el resultado producido por DBSCAN.